

O Codificador Limpo:

Um Código de Conduta para Programadores Profissionais



Capítulo 8: Estratégias de Testes

**Este material foi desenvolvido pelo grupo
PET/ADS do IFSP São Carlos**

Teste X Desenvolvedores

Os testes são um dos **pilares do desenvolvimento** de software, garantindo que o código produzido seja confiável.

Se os testes estão sendo feitos corretamente, alguns erros serão encontrados; porém, isso também indica que alguma parte do processo de desenvolvimento pode ser melhorada.

Os testes não devem ser vistos como um problema para os desenvolvedores, mas sim como um meio de garantir a **qualidade do sistema** e o cumprimento dos requisitos.

Especificação & Caracterização

O sistema deve possuir testes que sirvam como especificação do **comportamento esperado** , guiando os desenvolvedores sobre como o sistema deve funcionar.

Isso é feito por meio de **testes automatizados de aceitação** , que cobrem tanto o fluxo de sucesso quanto cenários de erro.

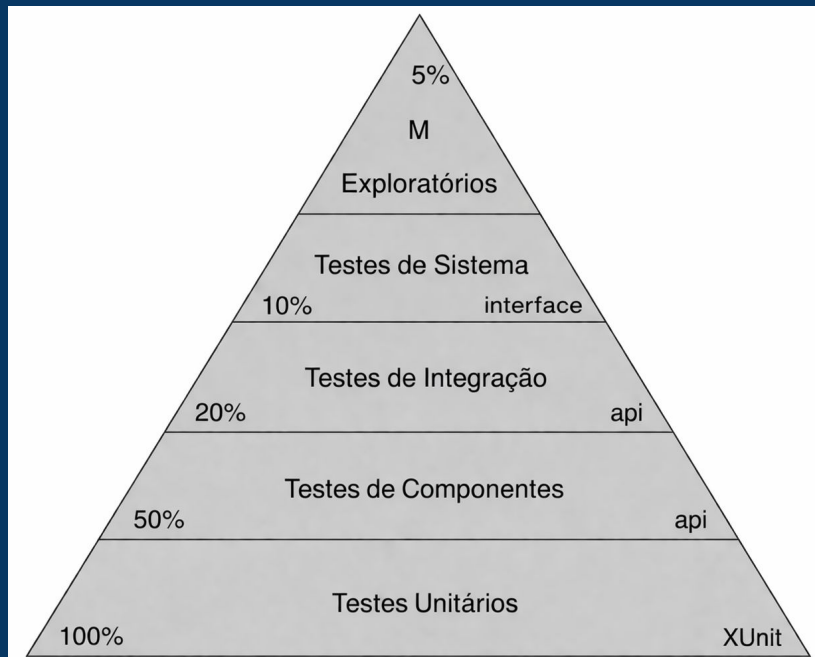
Além disso, os testes também têm o papel de caracterização, verificando o que de fato acontece com interações reais do sistema.

Pirâmide de testes

É uma boa prática utilizar diferentes níveis de testes, como **testes unitários, de aceitação e de integração**, para manter o sistema confiável.

No entanto, existem diferentes níveis que garantem a qualidade do sistema como um todo.

A **pirâmide de testes** representa a distribuição ideal desses testes em uma equipe de desenvolvimento:



Testes Unitários

Os testes unitários são desenvolvidos pela equipe de desenvolvimento e fazem parte da **Integração Contínua** .

Idealmente, são escritos antes do código de produção, **definindo o comportamento esperado** do que será implementado.

O objetivo é atingir uma cobertura o mais próxima possível de 100%, sendo uma média realista algo em torno de 90%.

Essa cobertura **deve ser real** , com testes que de fato verificam o comportamento do código, e não apenas executam trechos sem validação.

Testes de Componentes

Os testes de componentes fazem parte dos **testes de aceitação** , consistindo em isolar um componente que representa uma regra de negócio.

A partir disso, são inseridos dados de entrada e verificado se o resultado corresponde ao esperado.

Geralmente, são implementados em ferramentas específicas (Cucumber, JBehave, etc.), permitindo que outras áreas, como o negócio, consigam ler e **compreender os testes** .

O foco principal do teste é no fluxo principal (happy path), pois os cenários de erro, em sua maioria, já são cobertos pelos testes unitários.

Testes de Integração

Os testes de integração são aplicados em **partes maiores** , agrupando componentes e verificando a comunicação entre eles.

Diferente do anterior, não é focado em regras de negócio, e sim como as diferentes partes do sistema **trabalham em conjunto** , garantindo que a integração do sistema esteja correta.

Por serem testes mais complexos e demorados, geralmente não fazem parte da Integração Contínua, sendo executados periodicamente.

Testes de Sistema

Os testes de sistema validam o sistema **como um todo** , com todos os componentes integrados, incluindo também testes de desempenho.

O foco não está nas regras de negócio ou no comportamento individual, mas no atendimento de requisitos de qualidade, como desempenho, confiabilidade e responsividade.

Testes exploratórios manuais

Diferente dos demais, esses testes **não são automatizados** , sendo realizados por pessoas com o objetivo de explorar comportamentos não previstos anteriormente.

Não há uma meta de cobertura, pois o objetivo não é garantir os requisitos.

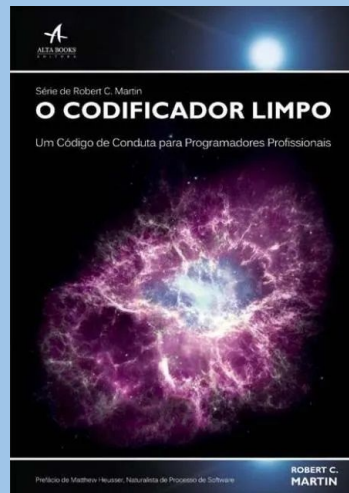
A ideia é submeter o sistema a uma “prova de fogo”, avaliando seu comportamento em situações **reais e inesperadas** .

Sobre o autor

Robert C. Martin é um engenheiro de software e consultor norte-americano com décadas de experiência na indústria. Conhecido como “Uncle Bob”, é um dos autores do Manifesto Ágil e escreveu livros influentes sobre boas práticas de programação, como O Codificador Limpo, baseados em sua experiência ensinando desenvolvedores a produzir código de qualidade.

Este conjunto de slides foi elaborado a partir da obra:

MARTIN, Robert C. **O codificador limpo:**
Um código de conduta para programadores
profissionais. 1. ed. Rio de Janeiro: Alta
Books, 2012.



A equipe



Érick Rosdachimo, Autor

Aluno do quarto período, é bolsista do PET/ADS desde abril de 2025.



Felipe, Revisor

Aluno do quarto período, é bolsista do PET/ADS desde dezembro de 2025.



Lucas Oliveira, Tutor

Professor de Computação, é tutor do PET/ADS desde janeiro de 2023.